

RK TEA SHOP – PROFESSIONAL FULL- STACK + AWS PROJECT PROMPT

Build a modern, professional, responsive tea-shop website called **RK TEA SHOP**.

The website should look like a real local tea shop's production website, not like a basic college demo. The design must be clean, attractive, fast, mobile-friendly, and easy for customers to use.

1. Business Information

Field	Details
Business Name	RK TEA SHOP

Field	Details
Phone	9843938881
Email	rengasamydharneesh@gmail.com
Opening Hours	5:00 AM – 8:00 PM

Use these details consistently throughout the website.

2. Technology Stack

Frontend

- React.js
- HTML5
- CSS3
- JavaScript
- Responsive design
- Modern reusable components

Backend

- Node.js
- Express.js
- REST API

Database

Use **Amazon RDS** for production database storage.

Store suitable data such as:

- Products
- Categories
- Customer enquiries
- Orders/enquiries
- Contact information
- Admin data

AWS Services

Use AWS professionally:

- **Amazon EC2** – application hosting
- **Amazon RDS** – database
- **Amazon S3** – images/static assets
- **IAM** – secure AWS permissions

- **CloudWatch** — monitoring and logs
- **Route 53** — DNS
- **AWS Certificate Manager** — SSL certificate
- **Application Load Balancer** where appropriate

Do not hard-code AWS credentials. Use IAM roles whenever possible.

3. Website Design

Create a premium tea-shop visual style. The website should include:

- Attractive hero section
- Tea/coffee shop imagery
- Clean typography
- Professional navigation bar
- Responsive mobile menu
- Modern cards
- Smooth hover effects
- Subtle animations
- Proper spacing

- Clear call-to-action buttons
- Consistent design throughout the website

Avoid excessive animations or flashy effects.
The website should load quickly even on mobile devices.

4. Homepage

Navigation

Include:

- RK TEA SHOP logo/name
- Home
- About
- Menu
- Gallery
- Contact
- Order / Enquiry button

Hero Section

Display:

RK TEA SHOP

Tagline: *"Fresh Tea. Great Taste. Every Day."*

Buttons:

- View Menu
- Contact Us

Add a high-quality tea-shop visual.

5. About Section

Create an attractive About section explaining the shop.

RK Tea Shop is a local tea shop focused on serving fresh tea, coffee, snacks, and everyday refreshments with good taste and friendly service.

Do not make unsupported claims such as awards, years of experience, certifications, or customer numbers unless they are provided separately.

6. Menu Section

Create a professional digital menu organized into categories:

Tea

Item	Notes
Tea	Classic
Ginger Tea	
Lemon Tea	
Special Tea	

Coffee

Item	Notes
Coffee	Classic
Filter Coffee	
Special Coffee	

Refreshments

- Cool Drinks

- Milk
- Other available refreshments

Snacks

- Vadai
- Bonda
- Bajji
- Biscuits
- Other snacks

Use realistic placeholder prices if actual prices are not supplied. Clearly label placeholder prices so they can easily be changed later.

Each menu item should have:

- Product name
- Short description
- Price
- Image
- Add/Enquire button where appropriate

7. Menu Search and Filtering

Add a customer-friendly menu system.

Customers should be able to:

- Search products
- Filter by category
- View product details
- Quickly find tea, coffee, or snacks

The filtering should work smoothly without refreshing the entire page.

8. Customer Enquiry

Create a simple enquiry form with fields:

- Name
- Phone
- Email
- Message

Add validation. Show clear success/error messages. Send enquiry data securely to the backend and store it in the database.

Never expose database credentials in frontend code.

9. Contact Section

Display:

Field	Details
Business	RK TEA SHOP
Phone	9843938881
Email	rengasamydharneesh@gmail.com
Opening Hours	5:00 AM – 8:00 PM

Add:

- Call button
- Email button
- Google Maps/location section
- Contact form

Do not invent the shop's exact address.
Provide a clearly marked placeholder until
the actual address is supplied.

10. Gallery

Create a responsive image gallery showing:

- Tea
- Coffee
- Snacks
- Shop atmosphere
- Food preparation
- Customer-friendly shop environment

Use optimized images. Images should be stored in Amazon S3 in the production version.

11. Backend API

Create clean REST APIs:

```
GET    /api/products
GET    /api/products/:id
POST   /api/products
PUT    /api/products/:id
DELETE /api/products/:id

POST   /api/enquiries
GET    /api/enquiries
```

```
GET    /api/health
```

- Return proper HTTP status codes and JSON responses
 - Add centralized error handling
 - Validate incoming data
-

12. Admin Dashboard

Create a simple protected admin dashboard.

Admin should be able to:

- Add products
- Edit products
- Delete products
- View enquiries
- Manage menu categories
- View basic application statistics

Do not expose admin functionality publicly without authentication.

13. Security

Implement basic production security:

- Environment variables
- Secure database credentials
- Input validation
- Authentication for admin routes
- Password hashing
- CORS configuration
- Rate limiting where appropriate
- HTTP security headers
- No hard-coded secrets
- No AWS access keys in source code

Create a `.env.example` file containing variable names only. Never commit `.env` to GitHub.

14. AWS Deployment Architecture

Application Flow

```
graph TD; Customer --> Route53[Route 53]; Route53 --> HTTPS[HTTPS / SSL]; HTTPS --> ALB[Application Load Balancer]; ALB --> EC2; EC2 --> Backend[Node.js / Express Backend]; Backend --> RDS[Amazon RDS];
```

Customer

↓

Route 53

↓

HTTPS / SSL

↓

Application Load Balancer

↓

EC2

↓

Node.js / Express Backend

↓

Amazon RDS

Static Assets

```
graph TD; Frontend --> S3[Amazon S3];
```

Frontend

↓

Amazon S3

Monitoring

```
graph TD; EC2 --> CloudWatch;
```

EC2

↓

CloudWatch



Logs + Monitoring

Use IAM roles instead of storing AWS credentials on EC2.

15. Docker

Containerize the application. Create:

- Dockerfile
- docker-compose.yml
- .dockerignore

Support separate containers where appropriate:

- Frontend
- Backend
- Database

For production, use Amazon RDS instead of running the database inside Docker.

16. CI/CD

Create a professional GitHub Actions pipeline:

```
graph TD; Developer --> GitHub; GitHub --> GitHub_Actions[GitHub Actions]; GitHub_Actions --> Build; Build --> Test; Test --> Docker_Build[Docker Build]; Docker_Build --> Security_Scan[Security Scan]; Security_Scan --> Amazon_ECR[Amazon ECR]; Amazon_ECR --> EC2_Deployment[EC2 Deployment];
```

Developer
↓
GitHub
↓
GitHub Actions
↓
Build
↓
Test
↓
Docker Build
↓
Security Scan
↓
Amazon ECR
↓
EC2 Deployment

Use GitHub OIDC with AWS IAM. Do not store long-lived AWS access keys inside GitHub Secrets when OIDC can be used.

17. Monitoring

Configure Amazon CloudWatch for:

- Application logs
- EC2 monitoring
- Backend errors
- CPU utilization
- Memory monitoring where configured
- Deployment troubleshooting

Create useful log messages in the backend.

18. SEO

Make the public website search-engine friendly. Include:

- Proper page title
- Meta description
- Semantic HTML
- Open Graph metadata
- Sitemap
- Robots.txt
- Mobile-friendly design
- Fast loading
- Descriptive image alt text

- Local-business structured data where appropriate
-

19. Performance

Optimize the website for production:

- Compress images
- Lazy-load gallery images
- Minimize unnecessary JavaScript
- Use reusable React components
- Enable appropriate caching
- Avoid unnecessary API requests
- Optimize database queries
- Use production builds

Target a fast and responsive mobile experience.

20. Project Structure

```
rk-tea-shop/  
|
```

```
|— frontend/
|   |— src/
|   |— public/
|   |— package.json
|   └─ Dockerfile
|
|— backend/
|   |— src/
|   |   |— controllers/
|   |   |— routes/
|   |   |— models/
|   |   |— middleware/
|   |   └─ config/
|   |— package.json
|   └─ Dockerfile
|
|— .github/
|   └─ workflows/
|
|— docker-compose.yml
|— .gitignore
|— .env.example
└─ README.md
```

Keep frontend and backend code clearly separated.

21. GitHub README

Create a professional README containing:

- Project overview
- Features
- Technology stack
- Architecture diagram
- AWS services
- Local setup
- Environment variables
- API documentation
- Docker instructions
- Deployment steps
- Screenshots
- Security practices
- Future improvements

Mention that this project demonstrates:

React + Node.js + Express + REST API +
RDS + S3 + EC2 + Docker + GitHub Actions
+ AWS IAM + CloudWatch

22. Final Quality Requirements

The finished application must:

- ☒ Look professional
- ☒ Be fully responsive
- ☒ Work on mobile and desktop
- ☒ Have clean code
- ☒ Have reusable components
- ☒ Have working backend APIs
- ☒ Store data correctly
- ☒ Handle errors properly
- ☒ Protect admin functionality
- ☒ Use secure AWS architecture
- ☒ Avoid hard-coded credentials
- ☒ Be Docker-ready
- ☒ Be GitHub-ready
- ☒ Be deployable to AWS
- ☒ Include clear documentation

The final result should be suitable as a **college portfolio project**, **AWS/cloud project**, and **full-stack developer project**.

Build the project step-by-step, testing each major feature before moving to the next stage. Do not skip configuration steps or assume that AWS resources already exist.